

Sistema de análisis de fotogramas con detección y reconocimiento facial

SOA: 2026

Introducción

El presente trabajo propone una arquitectura de servicios backend orientada al procesamiento de imágenes mediante modelos de inferencia. La solución permite recibir fotogramas, ejecutar modelos de detección, almacenar los resultados obtenidos y consultar posteriormente la información procesada mediante criterios flexibles.

En una primera etapa, el sistema se plantea sin interfaz gráfica, exponiendo sus funcionalidades mediante APIs REST que podrán ser consumidas desde herramientas genéricas como Postman, curl o scripts automatizados. Esta decisión permite validar el funcionamiento de los servicios principales de forma desacoplada, priorizando la correcta definición de entradas, salidas, persistencia y trazabilidad de los datos.

La solución contempla además una extensión basada en reconocimiento facial, permitiendo registrar personas, asociar imágenes a dichas personas, generar embeddings faciales y reconocer individuos en nuevas imágenes cuando se supere un umbral mínimo de confianza.

Finalmente, se proyecta una segunda entrega en la que se incorporarán componentes adicionales de interfaz gráfica, seguridad mediante Keycloak, autenticación biométrica opcional y monitoreo del sistema mediante Telegraf y Grafana.

Caso de uso general

Contexto

Una organización necesita procesar imágenes capturadas desde distintas fuentes (por ejemplo: cámaras fijas, dispositivos móviles o sistemas embebidos) con el objetivo de:

- Detectar objetos o entidades dentro de los fotogramas.
- Almacenar dicha información junto con metadatos geográficos.
- Permitir consultas posteriores sobre los datos procesados.
- Identificar personas específicas dentro de las imágenes utilizando reconocimiento facial.

En esta primera etapa, el sistema no contará con interfaz gráfica (frontend). Toda la interacción se realizará mediante clientes genéricos de API (por ejemplo: Postman o curl).

Flujo general del caso de uso

1. Selección de modelo de detección

- Un cliente consulta los modelos disponibles mediante S1.
- Selecciona uno de los modelos para ejecutar inferencia.

2. Procesamiento de fotograma

- El cliente envía un fotograma junto con metadatos (latitud, longitud, etc.) y el modelo seleccionado al Servicio 2.
- El sistema ejecuta la detección.
- Se almacenan:
 - La imagen
 - Los metadatos
 - Las detecciones
- Todo queda vinculado a un identificador único.

3. Consulta posterior

- El cliente puede consultar los datos mediante S4 utilizando filtros:
 - Ubicación (lat/lon)
 - Clases detectadas
 - Metadatos adicionales
- Obtiene una lista de resultados con referencias a las imágenes.

4. Recuperación de imagen

- A partir del [imageURL](#), el cliente consume el Servicio 3 para obtener la imagen completa o en formato thumbnail.

Extensión: reconocimiento facial

5. Registro de personas

- Se crean registros de personas en el sistema (nombre, apellido, email, etc.).

6. Carga de imágenes y generación de embeddings

- Se asocian imágenes a una persona mediante el servicio correspondiente.
- El sistema genera embeddings faciales y los almacena.

7. Reconocimiento en nuevas imágenes

- Se envía una imagen al servicio de reconocimiento facial.
- El sistema:
 - Detecta rostro
 - Genera embedding
 - Compara contra los almacenados
- Si supera el umbral de confianza:
 - Retorna la persona identificada
- Caso contrario:
 - Retorna resultado negativo

Objetivo del sistema

Permitir la construcción de una plataforma backend capaz de:

- Procesar imágenes de forma desacoplada
- Persistir resultados de inferencia
- Consultar información de forma flexible
- Incorporar identificación de personas sin modificar el flujo base

Caso de uso general – Sistema de análisis de fotogramas con detección y reconocimiento facial

El sistema permite procesar imágenes (fotogramas), detectar objetos, almacenar los resultados junto con metadatos y, adicionalmente, identificar personas mediante reconocimiento facial. En esta etapa no existe frontend. Todos los servicios son APIs REST y se consumen mediante clientes genéricos (ej.: Postman, curl).



Objetivo

Proveer un conjunto de servicios que permitan:

- Ejecutar inferencia de detección sobre fotogramas.
- Almacenar y consultar los resultados de detección junto con metadatos.
- Registrar personas y generar embeddings faciales.
- Reconocer personas en nuevas imágenes.

Actores



Cliente API

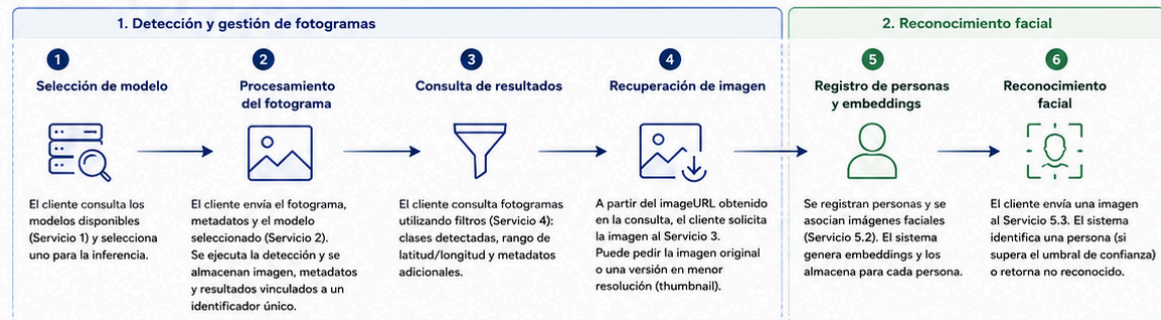
(usuarios o sistemas externos que consumen los servicios mediante HTTP)



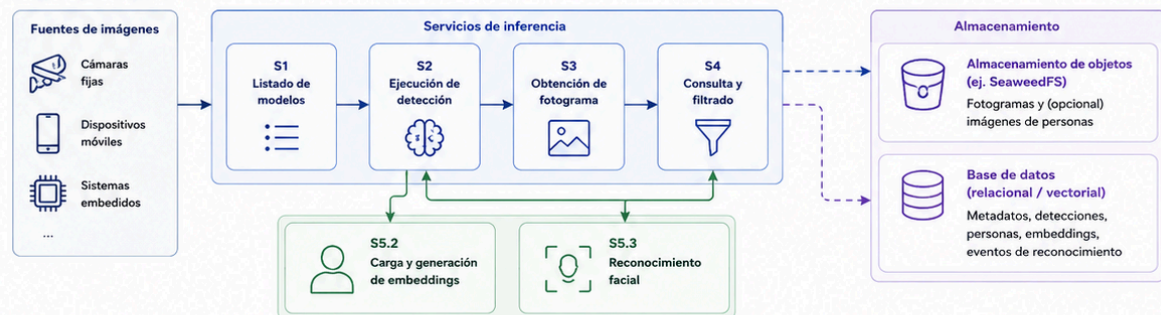
Servicios Backend

Conjunto de servicios de inferencia, almacenamiento y consulta.

Flujo general del caso de uso



Visión de alto nivel



Ejemplo de escenario completo

1. El cliente obtiene la lista de modelos disponibles (S1).
2. Selecciona un modelo y envía un fotograma con metadatos (lat, lon y otros) al servicio de detección (S2).
3. El sistema ejecuta inferencia, almacena todo y retorna el identificador del fotograma/proceso.
4. El cliente consulta fotogramas filtrando por ubicación y clase detectada (S4).
5. Obtiene el imageUrl y solicita la imagen al servicio correspondiente (S3).
6. (Opcional) Para reconocimiento facial:
 - a. Registra personas y carga imágenes para generar embeddings (S5.2).
 - b. Envía una nueva imagen al servicio de reconocimiento (S5.3) y obtiene la persona identificada (si aplica).

Características clave

- ✓ Servicios 100% backend, accesibles vía API REST.
- ✓ Sin dependencia de interfaz gráfica.
- ✓ Almacenamiento escalable y desacoplado.
- ✓ Esquema flexible para metadatos y atributos de personas.
- ✓ Identificación mediante umbral de confianza configurable.
- ✓ Todos los datos vinculados por identificadores únicos.

Resumen de servicios

A continuación se presenta una vista consolidada de los servicios definidos en el sistema, incluyendo sus principales responsabilidades, entradas y salidas:

Servicio	Nombre	Entrada principal	Salida principal	Persistencia
S1	Listado de modelos	-	Lista de modelos disponibles	No aplica
S2	Ejecución de detección	Fotograma + metadatos + modelo	ID del proceso / resultados de detección	Imagen (objeto), metadatos (BD), detecciones (BD)
S3	Obtención de fotograma	<code>frameId</code>	Imagen (original o thumbnail)	Lectura desde almacenamiento de objetos
S4	Consulta y filtrado	Filtros (clases, lat/lon, metadatos)	Lista de resultados con <code>imageURL</code> , metadata y detecciones	Lectura desde base de datos
S5.1	Gestión de personas	Datos de persona	Registro de persona	Base de datos
S5.2	Generación de embeddings	<code>personId</code> + imágenes	Embeddings generados	BD (embeddings) + opcional objetos (imágenes)
S5.3	Reconocimiento facial	Imagen + threshold (opcional)	Persona identificada (si aplica)	Lectura de embeddings

Notas

- Todos los servicios se exponen mediante APIs REST.
- Los datos generados se vinculan mediante identificadores únicos (`frameId`, `personId`, etc.).

Identificadores principales

El sistema utiliza identificadores únicos para mantener la trazabilidad entre imágenes, metadatos, resultados de detección, personas y embeddings faciales.

Identificador	Descripción
<code>frameId</code>	Identifica de forma única un fotograma procesado por el sistema.
<code>modelId</code>	Identifica el modelo de inferencia utilizado para procesar un fotograma.
<code>detectionId</code>	Identifica el resultado de una ejecución de detección.
<code>personId</code>	Identifica de forma única una persona registrada.
<code>embeddingId</code>	Identifica una representación facial generada para una persona.
<code>recognitionId</code>	Identifica un intento o resultado de reconocimiento facial.

Todos los datos persistidos deben poder relacionarse mediante estos identificadores, garantizando trazabilidad, consistencia y posibilidad de consulta posterior.

Backend – Servicios de inferencia (detección)

El sistema debe componerse de un conjunto de servicios orientados exclusivamente a la ejecución de modelos de detección sobre fotogramas, junto con la gestión, almacenamiento y consulta de los resultados.

1. Gestión de modelos

El sistema debe disponer de un repositorio de modelos almacenados en una carpeta accesible por los servicios de inferencia.

S1 – Listado de modelos disponibles

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: GET
URL: /models

Descripción: Retorna la lista de modelos disponibles

Expone un endpoint que retorna la lista de modelos disponibles en dicha carpeta.

Ejemplo de respuesta:

```
["yolo11n.pt", "yolo11s.pt"]
```

2. Procesamiento e inferencia

S2 – Ejecución de modelos de detección

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: POST
URL: /detections

Descripción: Ejecuta un modelo de detección sobre un fotograma y persiste los resultados

Entrada

- Un fotograma (imagen)
- Metadatos asociados
- Identificador del modelo a ejecutar(`modelId`) (debe pertenecer al conjunto retornado por S1)

Funcionalidad

- Ejecuta el modelo seleccionado sobre el fotograma recibido.
- Persiste la información generada según el siguiente esquema:

Almacenamiento

- **Fotograma:**
Almacenado en un sistema de almacenamiento de objetos (por ejemplo, SeaweedFS).
Se le asigna un identificador único (**frameId**).
- **Metadatos:**
Almacenados en una base de datos relacional.
Asociados al **frameId**.
 - Deben incluir como mínimo:
 - Latitud
 - Longitud
 - Se recomienda un esquema flexible mediante JSON para soportar metadatos adicionales variables.
- **Resultados de detección:**
 - Almacenados en la base de datos.
 - Asociados al **frameId**.
 - Formato sugerido: JSON con clases detectadas, bounding boxes, scores, etc.

Identificación

- Todos los elementos generados en el proceso (fotograma, metadatos y resultados de detección) deben estar vinculados mediante el identificador único **frameId**, permitiendo su trazabilidad y posterior consulta.

Consideración de concurrencia (opcional)

- Se valorará que la ejecución del modelo se procese de forma asíncrona (por ejemplo, mediante hilos secundarios o colas de trabajo) para mejorar la concurrencia del sistema.

3. Recuperación de datos

S3 – Obtención de fotograma

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: GET

URL: /frames/{frameId}

Descripción: Retorna la imagen asociada a un identificador

Parámetros

- **frameId**: identificador único del fotograma
- **thumbnail** (opcional): booleano
 - **true** → retorna una versión de menor resolución
 - **false** o ausente → retorna la imagen original

Entrada

- Identificador único del fotograma (**frameId**)

Salida

- Imagen asociada al **frameId**

Consideración

- El parámetro opcional **thumbnail** permite solicitar una versión de menor resolución de la imagen, con el objetivo de optimizar la transferencia y visualización.

4. Consulta y filtrado

S4 – Servicio de consulta

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: GET

URL: /frames/search

Descripción

Permite recuperar fotogramas previamente procesados en base a criterios de filtrado.

Parámetros de consulta (filtros)

- Clases detectadas
- Metadatos adicionales (estructura flexible)
- Latitud y longitud (obligatorios, con posibilidad de definir rangos)

Salida

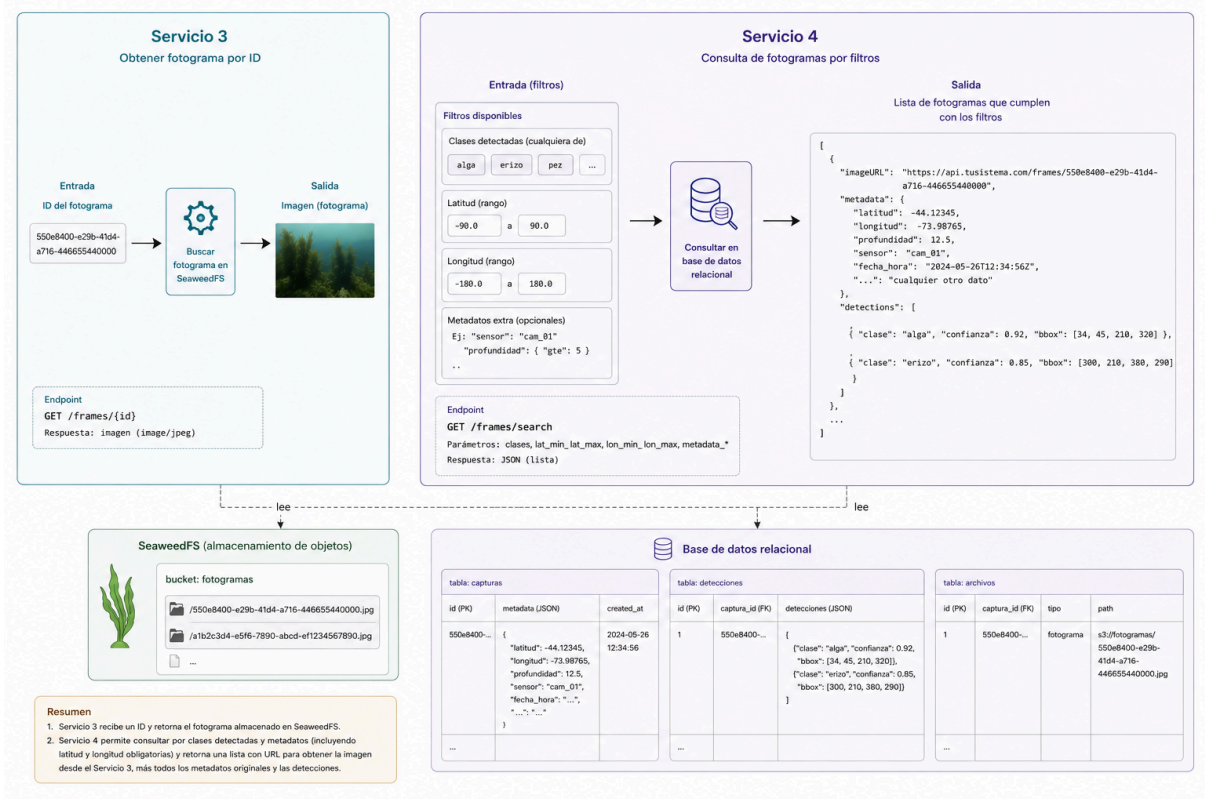
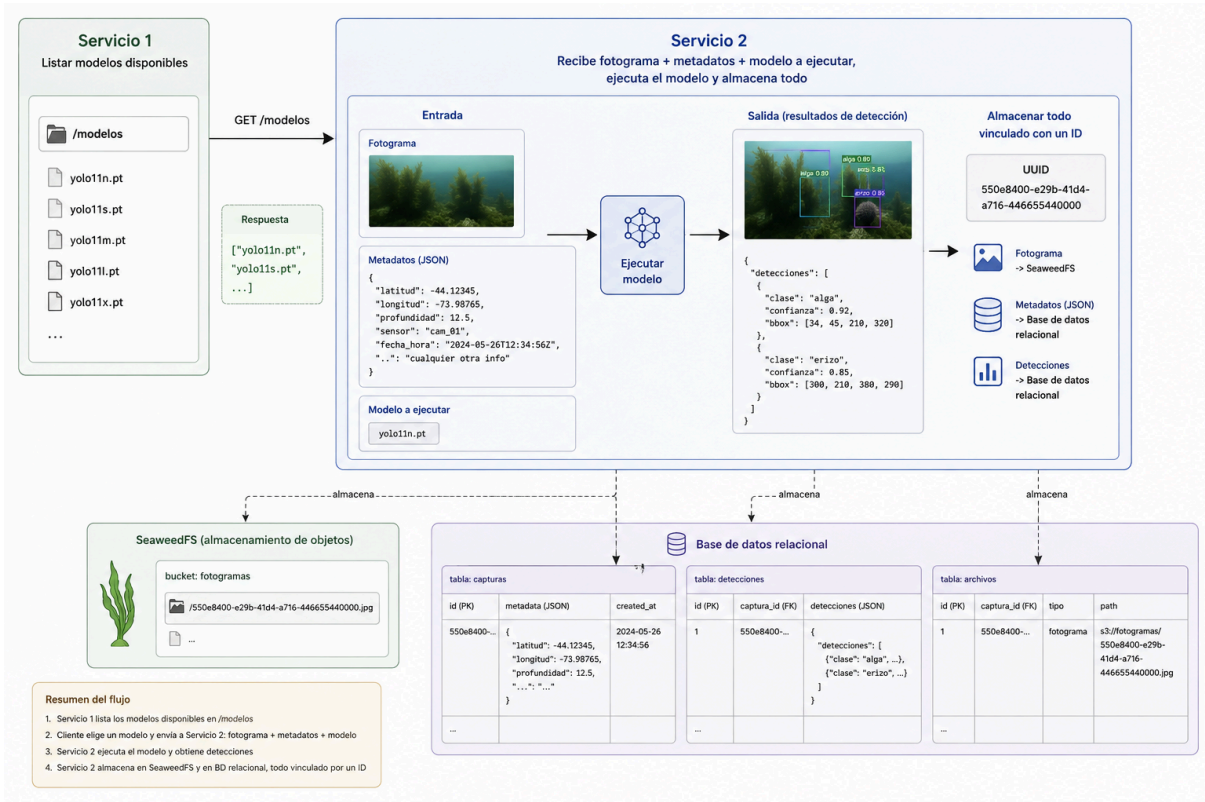
Retorna una lista de resultados con el siguiente formato:

```
[
  {
    "frameId": "uuid",
    "imageUrl": "URL para obtener la imagen mediante S3",
    "metadata": {
      "...": "metadatos originales completos"
    },
    "detections": [
      {
        "...": "datos de detección generados por el modelo"
      }
    ]
  }
]
```

Consideraciones adicionales

Este servicio debe integrarse con la base de datos relacional para resolver consultas eficientes sobre metadatos y resultados de detección, manteniendo la coherencia con los identificadores únicos generados en el proceso de ingestión.

Bocetos generales de servicios y flujo de datos (guía)



Backend – Servicios de inferencia (se agrega reconocimiento facial)

El sistema se extiende incorporando capacidades de reconocimiento facial, permitiendo registrar personas, generar embeddings y realizar identificación a partir de imágenes.

5. Servicio de reconocimiento facial

5.1. Modelo de datos de personas

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: POST

URL: /persons

Descripción: Crea una nueva persona

(Opcional)

Método: GET

URL: /persons/{personId}

Descripción: Obtiene información de una persona

Entidad: Persona

Estructura mínima

- `personId` (UUID)
- `nombre`
- `apellido`
- `email`

Extensibilidad

Se permite agregar información adicional mediante un campo JSON:

```
{
  "personId": "uuid",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan@mail.com",
  "extra": {
    "...": "atributos adicionales"
  }
}
```

S5.2 – Carga y generación de embeddings

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: POST

URL: /persons/{personId}/embeddings

Descripción: permite asociar imágenes a una persona y generar las representaciones faciales (embeddings).

Entrada

```
{
  "personId": "uuid",
  "images": ["imagen1", "imagen2"]
}
```

Proceso

- Validación de la persona (**personId**)
- Detección de rostro en cada imagen
- Generación de embeddings
- Asociación de los embeddings con la persona

Salida

```
{
  "personId": "uuid",
  "processedImages": 5,
  "validEmbeddings": 4,
  "rejectedImages": 1
}
```

Almacenamiento

Imágenes (opcional, recomendado):

- Almacenamiento en sistema de objetos (ej. SeaweedFS), consistente con el almacenamiento de fotogramas del sistema principal.

Embeddings:

- Base de datos (relacional o vectorial).

```
{
  "embeddingId": "uuid",
  "personId": "uuid",
  "vector": [...]
}
```

Consideración de concurrencia (opcional)

Se recomienda procesar la generación de embeddings de forma asíncrona (proceso secundario o cola), para mejorar la capacidad de respuesta del sistema.

S5.3 – Reconocimiento facial

Todos los servicios se exponen mediante APIs REST utilizando formato JSON.

Endpoint

Método: POST

URL: /face-recognition

Descripción: Identifica una persona a partir de una imagen

Entrada

```
{
  "image": "imagen",
  "threshold": 0.8
}
```

Proceso

- Detección de rostro
- Generación de embedding
- Comparación contra embeddings almacenados
- Selección del mejor candidato

Salida (reconocido)

```
{
  "personId": "uuid",
  "nombre": "Juan",
  "apellido": "Pérez",
  "confidence": 0.87
}
```

Salida (no reconocido)

```
{
  "personId": null,
  "confidence": 0.45
}
```

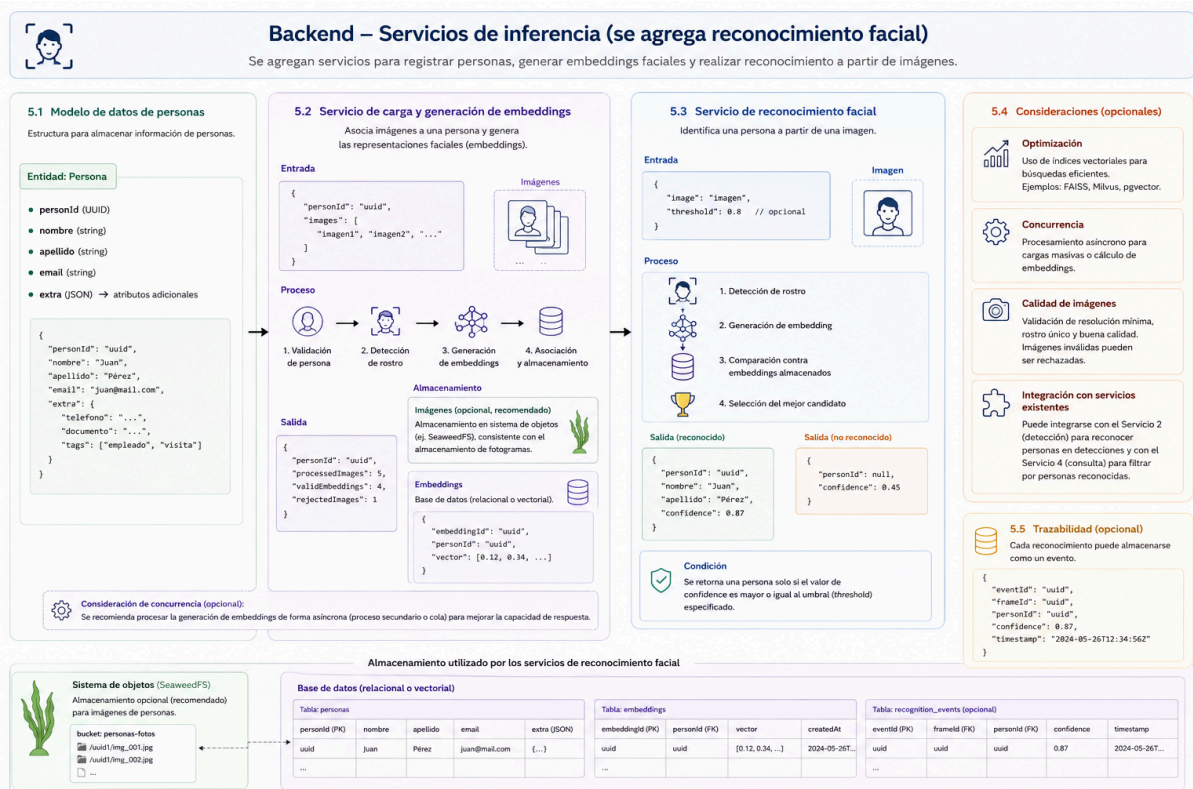
Condición

Se retorna una persona únicamente si el nivel de confianza (**confidence**) supera el umbral definido (**threshold**).

5.4. Consideraciones (opcionales)

- Optimización mediante índices vectoriales (FAISS, Milvus, pgvector).
- Procesamiento concurrente para cargas masivas.
- Validación de calidad de imágenes (resolución, rostro único).
- Integración con servicios de detección para enriquecer resultados.

Bocetos generales de servicios y flujo de datos (guía)



6. Evolución del sistema – Segunda entrega

La segunda entrega tiene como objetivo extender la arquitectura actual incorporando:

- Interfaz gráfica de usuario (UI)
- Mecanismos de autenticación y autorización
- Capacidades de monitoreo y observabilidad

6.1. Incorporación de interfaz gráfica (Frontend)

Se desarrollará una interfaz gráfica que permitirá interactuar con los servicios backend de forma más accesible.

Objetivos

- Facilitar la carga de imágenes (fotogramas)
- Visualizar resultados de detección
- Consultar datos filtrados
- Administrar personas para reconocimiento facial

Funcionalidades esperadas

- Selección de modelos de detección
- Envío de imágenes con metadatos
- Visualización de resultados (detecciones y reconocimiento)
- Consulta avanzada mediante filtros (clases, ubicación, metadatos)
- Gestión de personas:
 - Alta / baja / modificación
 - Carga de imágenes para generación de embeddings

Consideraciones

- El frontend será un cliente desacoplado que consumirá exclusivamente APIs REST existentes.
- No se incorporará lógica de negocio en la interfaz.



6.2. Autenticación biométrica (opcional)

Se evaluará la incorporación de autenticación basada en reconocimiento facial.

Descripción

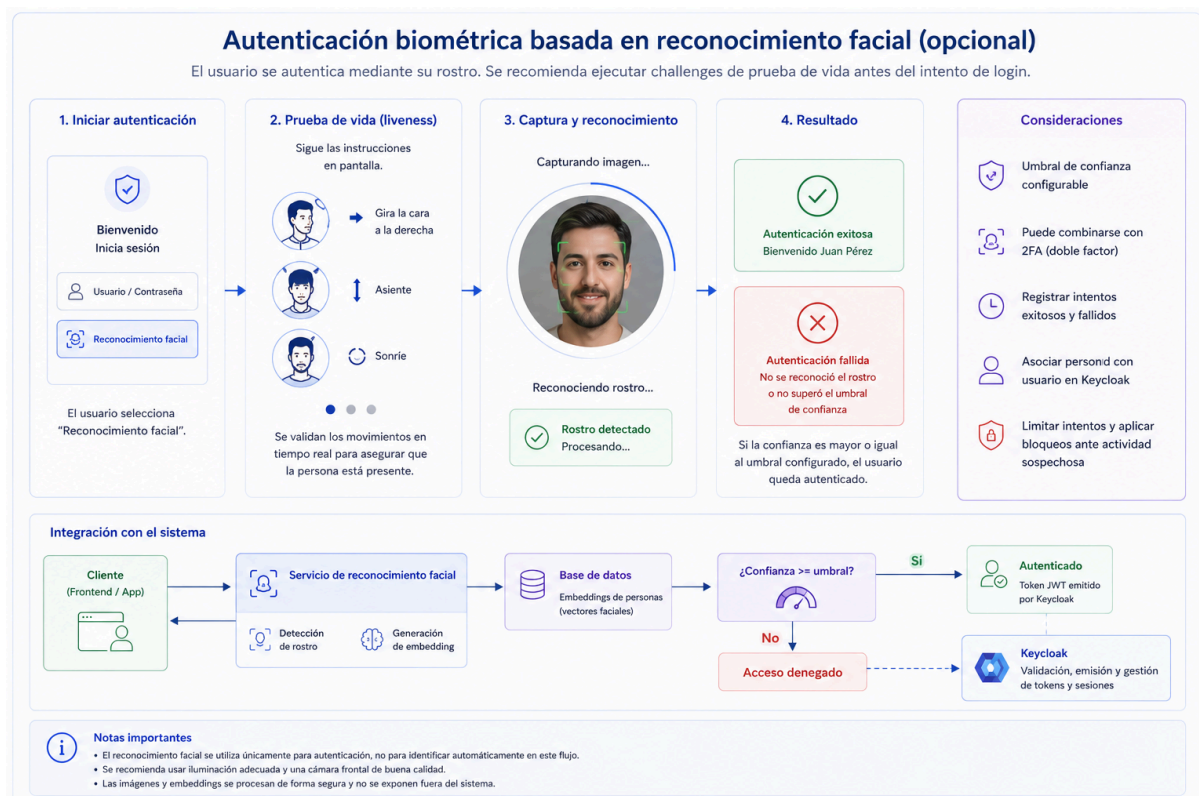
Permite autenticar usuarios mediante una imagen, utilizando los servicios de reconocimiento facial ya definidos.

Flujo propuesto

1. El cliente envía una imagen al sistema.
2. Se ejecuta el servicio de reconocimiento facial.
3. Si se identifica una persona válida:
 - Se considera autenticado el usuario
4. En caso contrario:
 - Se rechaza el acceso

Consideraciones

- El umbral de confianza debe ser configurable.
- Puede utilizarse como mecanismo complementario (ej. segundo factor).
- Su uso debe limitarse a contextos controlados.



6.3. Seguridad – Integración con Keycloak

Se incorporará un sistema de gestión de identidad y acceso utilizando **Keycloak**.

Objetivos

- Centralizar autenticación y autorización
- Proteger los servicios backend
- Gestionar usuarios, roles y permisos

Implementación

- Uso de OAuth2 / OpenID Connect
- Emisión de tokens JWT
- Validación de tokens en cada servicio backend

Protección de servicios

Todos los endpoints deberán requerir autenticación.

Ejemplo de roles

- **admin**: gestión completa del sistema
- **operator**: carga de imágenes y consultas
- **viewer**: acceso de solo lectura

Integración con reconocimiento facial

- Posibilidad de asociar **personId** con usuarios de Keycloak
- Uso del reconocimiento facial como mecanismo adicional de autenticación



6.4. Monitoreo y observabilidad

Se incorporarán herramientas para monitorear el estado del sistema y analizar su comportamiento.

Stack tecnológico

- **Telegraf**: recolección de métricas
- **Grafana**: visualización
- (Opcional) InfluxDB como almacenamiento de métricas

6.4.1. Métricas a recolectar

Servicios de detección

- Tiempo de procesamiento por inferencia
- Cantidad de requests
- Tasa de errores

Servicios de reconocimiento facial

- Tiempo de generación de embeddings
- Tiempo de comparación
- Cantidad de reconocimientos exitosos vs fallidos

Sistema general

- Uso de CPU y memoria
- Latencia de servicios
- Throughput

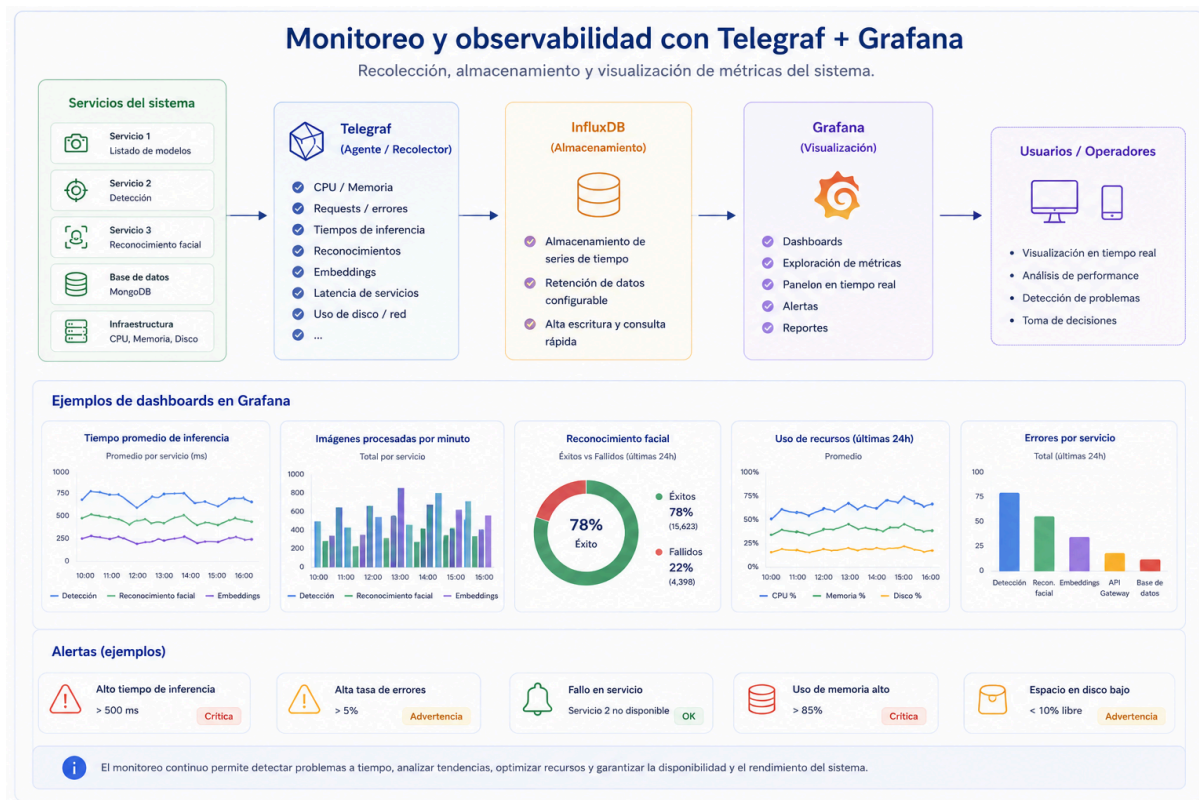
6.4.2. Visualización en Grafana

Se deberán construir dashboards que permitan:

- Monitoreo en tiempo real
- Detección de cuellos de botella
- Análisis de uso del sistema

Ejemplos de paneles

- Tiempo promedio de inferencia
- Cantidad de imágenes procesadas por minuto
- Ratio de reconocimiento exitoso
- Uso de recursos por servicio



6.5. Consideraciones de arquitectura

- Se mantiene una arquitectura desacoplada basada en servicios.
- El frontend consume APIs existentes sin alterar la lógica de negocio.
- Seguridad transversal mediante Keycloak.
- Observabilidad integrada como componente clave de operación.

6.6. Resultado esperado

Al finalizar la segunda entrega, el sistema contará con:

- Interfaz gráfica para interacción operativa
- Control de acceso seguro mediante autenticación y autorización
- Capacidades de monitoreo y análisis en tiempo real
- Mayor facilidad de uso sin comprometer la arquitectura backend existente

Criterios de evaluación (orientativos)

La evaluación del trabajo considerará de forma integral los siguientes aspectos:

Arquitectura y diseño

- Coherencia general de la solución.
- Correcta separación de responsabilidades entre servicios.
- Uso adecuado de una arquitectura desacoplada basada en APIs.

Funcionalidad

- Implementación completa de los servicios definidos.
- Correcto manejo de inputs y outputs.
- Flujo funcional de procesamiento, almacenamiento y consulta.

Persistencia y manejo de datos

- Diseño adecuado del almacenamiento (objetos, base relacional, JSON).
- Correcta vinculación de entidades mediante identificadores únicos.
- Flexibilidad para manejar metadatos variables.

Reconocimiento facial

- Implementación del flujo de generación de embeddings.
- Correcto funcionamiento del reconocimiento basado en umbral de confianza.
- Integración con el resto del sistema.

Seguridad (si aplica)

- Uso adecuado de autenticación y autorización.
- Integración con herramientas como Keycloak.
- Protección de endpoints.

Monitoreo (si aplica)

- Recolección de métricas relevantes.
- Visualización clara mediante dashboards.
- Capacidad de análisis del comportamiento del sistema.

Calidad general

- Claridad en la documentación.
- Consistencia en el diseño de APIs.
- Manejo adecuado de errores.
- Código mantenible y bien estructurado.

Extras valorados

- Uso de procesamiento asíncrono.
- Consideraciones de escalabilidad.
- Extensiones o mejoras sobre la solución base.

Primera entrega:

Hasta punto 5.4 inclusive

Fecha de entrega: 9/6/2026

Restricción: ausencia de frontend

En esta etapa:

- **No existe interfaz gráfica**
- Todos los servicios deben exponerse como APIs REST
- Deben poder ser consumidos mediante:
 - Postman
 - curl
 - scripts automatizados

Segunda entrega (final):

Desde punto 6) completo y haber salvado posibles observaciones anteriores